Zustand

Table of Contents

Zustand

```TEXT
npm install zustand
```

Store — The Heart of Zustand

A store holds all state and actions together.

```JS
import { create } from 'zustand';

const useStore = create((set, get) => ({
    // State
    count: 0,
    name: 'John',

    // Actions
    increment: () => set((state) => ({ count: state.count + 1 })),
    decrement: () => set((state) => ({ count: state.count - 1 })),
    setName: (newName) => set({ name: newName }),
}));
```

Parameters:

- `set` — Function to update state
- `get` — Function to get current state
- `api` — Complete store API (includes set, get, subscribe, etc.)

Set — Updating State

JS

```
const useStore = create((set) => ({
    count: 0,
    users: [],

    increment: () => set((state) => ({ count: state.count + 1 })),

    addUser: (user) => set((state) => ({
        users: [...state.users, user],
    })),

    removeUser: (userId) => set((state) => ({
        users: state.users.filter(u => u.id !== userId),
    })),
}));
```

Immer — Making Immutable Updates Simple

Without Immer (verbose spread operations):

JS

```
const updateNestedState = (state) => ({
    ...state,
    user: {
        ...state.user,
        profile: {
            ...state.user.profile,
            address: {
                ...state.user.profile.address,
                city: 'New York',
            },
        },
    },
});
```

With Immer:

JS

```
import { produce } from 'immer';

const updateNestedState = produce((draft) => {
    draft.user.profile.address.city = 'New York';
});
```

Zustand + Immer middleware:

JS

```
import { create } from 'zustand';
import { immer } from 'zustand/middleware/immer';

const useStore = create(
    immer((set) => ({
        user: {
            profile: {
                name: 'John',
                address: { city: 'NYC', zip: '10001' },
            },
        },

        updateCity: (newCity) => set((state) => {
            state.user.profile.address.city = newCity;
        }),

        resetUser: () => set((state) => {
            state.user.profile.name = '';
            state.user.profile.address.city = '';
        }),
    }))
);
```

Get — Accessing Current State

JS

```
const useStore = create((set, get) => ({
    count: 0,
    step: 2,
    history: [],

    increment: () => {
        const currentCount = get().count;
        const step = get().step;
        set({ count: currentCount + step });

        if (get().count > 10) {
            console.log('Count is high!');
        }
    },

    addToHistory: (action) => {
        const state = get();
        if (!state.history.includes(action)) {
            set({ history: [...state.history, action] });
        }
    },

    canIncrement: () => {
        return get().count < 100;
    },
}));
```

Selectors — Efficient State Access

Selectors extract specific pieces of state and prevent unnecessary re-renders.

JS

```
const useStore = create((set) => ({
    user: { id: 1, name: 'John', email: 'john@example.com' },
    posts: [],
    comments: [],
    loading: false,
}));
```

JSX

```jsx
// BAD — subscribes to everything
function BadComponent() {
    const state = useStore();
    return <div>{state.user.name}</div>;
}

// GOOD — subscribes only to user.name
function GoodComponent() {
    const userName = useStore((state) => state.user.name);
    return <div>{userName}</div>;
}

// Select multiple specific values
function Profile() {
    const [name, email] = useStore((state) => [
        state.user.name,
        state.user.email,
    ]);

    return <div>{name} — {email}</div>;
}
```

Actions

Synchronous Actions

JS

```js
const useStore = create((set, get) => ({
    items: [],
    selectedItem: null,

    addItem: (item) => set((state) => ({
        items: [...state.items, item],
    })),

    removeItem: (id) => set((state) => ({
        items: state.items.filter(item => item.id !== id),
    })),

    updateItem: (id, updates) => set((state) => ({
        items: state.items.map(item =>
            item.id === id ? { ...item, ...updates } : item
        ),
    })),
}));
```

Asynchronous Actions

JS

```
const useStore = create((set, get) => ({
    users: [],
    loading: false,
    error: null,

    fetchUsers: async () => {
        set({ loading: true, error: null });
        try {
            const response = await fetch('https://api.example.com/users')
            const data = await response.json();
            set({ users: data, loading: false });
        } catch (error) {
            set({ error: error.message, loading: false });
        }
    },

    createUser: async (userData) => {
        set({ loading: true });
        try {
            const response = await fetch('https://api.example.com/users',
                method: 'POST',
                body: JSON.stringify(userData),
            });
            const newUser = await response.json();
            set((state) => ({
                users: [...state.users, newUser],
                loading: false,
            }));
        } catch (error) {
            set({ error: error.message, loading: false });
        }
    },

    updateUserAndFetch: async (userId, updates) => {
        const { users } = get();

        // Optimistic update
        set({
            users: users.map(u =>
                u.id === userId ? { ...u, ...updates, updating: true } :
            ),
        });

        try {
            await fetch(`https://api.example.com/users/${userId}`, {
                method: 'PUT',
                body: JSON.stringify(updates),
```

```
            });

            set({
                users: users.map(u =>
                    u.id === userId ? { ...u, ...updates, updating: false
                ),
            });
        } catch (error) {
            // Rollback on error
            set({ users, error: error.message });
        }
    },
}));
```